

COMMON COURSE ASSIGNMENT

CSA08 – GENERATIVE AI & PROMPT ENGINEERING | Covering Large Language Models & Prompt-Driven Engineering Assistance

A. Assignment Information

Department	Computer Science and Engineering
Programme	B.E. / B.Tech – CSE
Course Code & Course Name	CSA08 – Generative AI and Prompt Engineering
Academic Year / Batch	2026 – 2027
Faculty Name	Dr.Chandravathi C
Assignment Title	Design and Implementation of PromptCraft – A Prompt-Driven Engineering Assistant using Large Language Models and Prompt Engineering
Date of Issue	21.08.2026
Date of Submission	03.09.2026
Maximum Marks	100
Course Outcome(s) – CO	CO1 & CO2
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze, L5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Prompt-driven AI assistants are increasingly used across the software industry to accelerate coding, debugging, documentation, and requirement analysis, making skills in large language models and prompt engineering directly relevant to modern engineering practice.

B. Assignment Problem / Challenge

This assignment requires students to design, implement, and evaluate an AI-powered engineering assistant that applies core Generative AI concepts — prompt design and engineering techniques (zero-shot, few-shot, and chain-of-thought prompting), large language model (LLM) API integration, and structured evaluation of model outputs — to a unified engineering-support platform called **Prompt Craft**. Students must integrate multiple prompting strategies, build task-specific engineering modules, deploy the system as a lightweight web application, validate output quality and reliability with sample test cases, and address basic responsible-AI considerations such as transparency of prompts, hallucination handling, and reproducibility of results.

C. Problem Statement

Scenario: “PromptCraft” – A Prompt-Driven Engineering Assistant

Many students and practicing engineers find it difficult to consistently get high-quality, reliable outputs from large language models without structured prompting. Your task is to design and implement an interactive engineering-support web application called **PromptCraft** that helps engineers construct effective prompts, apply prompt engineering techniques, and use an LLM to accelerate common software-engineering tasks through a unified interface.

The system shall integrate three major functional modules:

1. Prompt Engineering Workbench

Implement zero-shot, few-shot, and chain-of-thought prompting templates for common engineering tasks.

- Users should be able to enter a task description, select a prompting strategy, and optionally supply few-shot examples or context.
- The system must display the fully constructed prompt sent to the LLM together with the generated response, enabling students to see exactly how prompt structure affects output quality.

2. LLM-Powered Engineering Task Modules

Implement at least three engineering-assistant task modules, e.g., code generation & explanation, automated debugging assistance, and requirement-to-documentation generation.

- Users will enter their engineering artifact (a code snippet, an error/bug description, or a requirement statement) as input to the selected module.

- The system should show the underlying prompt template used for each module, the LLM output, and any post-processing applied, so that students can trace how the assistant arrived at its result.

3. Prompt Strategy Evaluation & Comparison

Implement comparison of at least three prompting strategies (e.g., zero-shot, few-shot, chain-of-thought) for the same task.

- Users will enter a representative engineering task and a set of reference/expected outputs for evaluation.
- The system should display response quality, response time / latency, and token usage for each strategy, allowing clear comparative analysis.

Design the application workflow so that user inputs (task descriptions, code snippets, bug reports, requirement statements) are processed through the corresponding prompt templates and LLM calls, and the results are presented as clear prompts, responses, and comparative metrics. Develop a clean, simple web interface using **Streamlit**. Provide 2–3 sample test cases for each module to demonstrate effectiveness. Students must use a real or simulated LLM API (e.g., OpenAI, Anthropic, or an open-source model) to generate responses.

Evaluate the application under suitable test cases and analyze the quality of LLM outputs, usability of the interface, and clarity of the prompt/response traces. The completed system should also briefly address responsible-AI considerations including transparency of prompts, hallucination and error handling, data privacy, and reproducibility of results.

D. Requirements and Constraints

- **Functional:** Provide interactive modules for a Prompt Engineering Workbench (with visible prompt construction and response display), LLM-Powered Engineering Task Modules (code generation, debugging assistance, and documentation generation), and Prompt Strategy Evaluation (quality, latency, and token-usage comparison). Accept user inputs as well as sample data. Produce comparative tables and prompt/response traces.
- **Interface:** Clean, simple, and user-friendly Streamlit web application with separate pages or clearly separated sections for each module.
- **Performance:** LLM calls must return results with acceptable latency for interactive classroom use and support clear comparative analysis across prompting strategies.
- **Technical constraint:** Use Python + Streamlit. No database or complex architecture required. Use any accessible LLM API or open-source model for prompt execution.

- **Reliability:** Prompt templates and task modules must behave consistently and reproducibly; intermediate prompts must be transparent; the system must handle malformed inputs and LLM API errors without crashing.
- **Resource constraint:** Project must be completable within the semester timeline using standard student computing resources.
- **Responsible Computing:** Ensure transparency of prompts and model limitations, reproducibility of results, mitigation of hallucinated or misleading outputs, and adherence to responsible-AI and data-privacy practices.
- **Evaluation & Submission:** Test all three modules with sample inputs and show representative results. Submit working Streamlit code, a short report containing sample prompts/outputs / screenshots, comparative analysis, and a GitHub repository link.

E. Student Work

1. Problem Understanding and Formulation

PromptCraft is designed as an interactive engineering-support application that helps students and engineers use Large Language Models effectively through structured prompt engineering. The system addresses the difficulty of obtaining consistent and useful LLM responses for common engineering tasks.

The main challenges are constructing effective prompts, selecting suitable prompting strategies, integrating an LLM API, evaluating generated responses, and maintaining transparency between the input, constructed prompt, and final output.

The expected outcomes are:

- Effective prompt construction using Zero-Shot, Few-Shot, and Chain-of-Thought prompting.
- LLM-assisted support for engineering tasks.
- Comparison of prompting strategies using quality, latency, and token usage.
- Transparent display of prompts and generated responses.
- Reliable handling of invalid inputs and API errors.

The major assumptions are that users have access to an LLM API, the application is used through a Streamlit interface, and engineering tasks are provided as text, code, error descriptions, or requirements.

The design follows the constraints specified in Section D, including Python and Streamlit implementation, acceptable response latency, reproducibility, error handling, prompt transparency, and responsible use of AI.

2. Application of Course Knowledge (CO2)

The implementation applies prompt engineering concepts and Large Language Model integration to engineering-support tasks.

Part I – Prompt Engineering Strategies

Zero-Shot Prompting:

The user provides the task directly without examples. The prompt contains the task, context, role, and expected response requirements.

Few-Shot Prompting:

The prompt contains example inputs and outputs before the actual task. These examples guide the LLM towards the required response structure and style.

Chain-of-Thought Prompting:

The system provides a structured reasoning format that encourages systematic analysis of the task, assumptions, technical considerations, solution, and verification.

Part II – LLM-Powered Engineering Tasks

The system implements three task-oriented modules:

1. **Code Generation and Explanation** – generates or explains programming solutions.
2. **Debugging Assistance** – analyses code and error descriptions and suggests corrections.
3. **Requirement-to-Documentation** – converts software requirements into structured technical documentation.

Each module uses a task-specific prompt template and displays the generated LLM response.

Part III – Prompt Strategy Evaluation

The same engineering task is processed using Zero-Shot, Few-Shot, and Chain-of-Thought strategies. The responses are evaluated using quality, latency, and token usage to identify the most suitable strategy for different engineering tasks.

3. Solution / Design / Methodology

The proposed PromptCraft system consists of three integrated modules. The Streamlit interface accepts the user's engineering task and allows selection of the required prompting strategy or engineering task module.

The processing workflow is:

User Input → Module Selection → Prompt Template → Prompt Construction → LLM API → Generated Response → Evaluation → Display

The Prompt Engineering Workbench constructs prompts dynamically according to the selected strategy. The engineering modules apply predefined templates for coding, debugging, and documentation tasks.

The evaluation module executes the same task with different prompting strategies and records response quality, latency, and token usage for comparison.

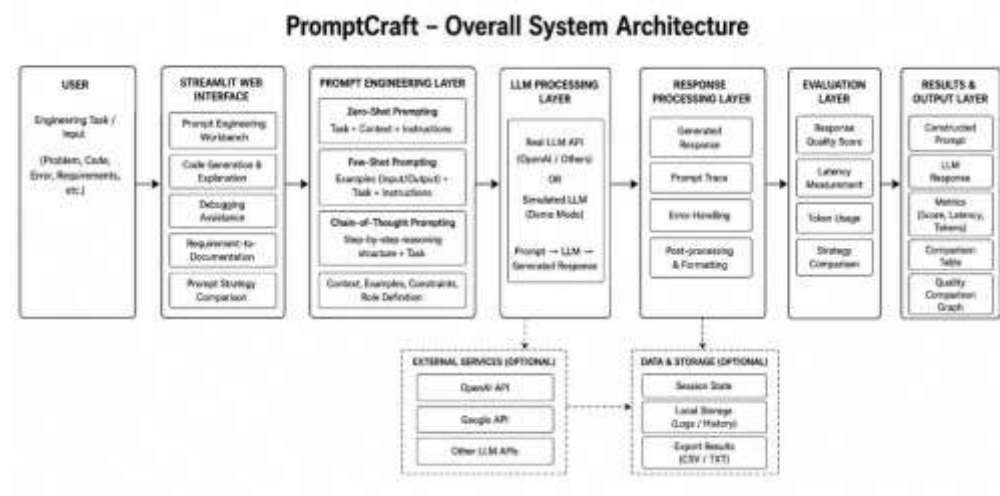


Fig 1: System Architecture

4. Use of Modern Tools

The application is implemented using **Python** and **Streamlit**. Python provides the programming environment for prompt construction, API communication, response processing, and evaluation. Streamlit provides the interactive web interface.

An accessible LLM API is integrated to generate engineering responses. The application uses structured prompt templates to implement Zero-Shot, Few-Shot, and Chain-of-Thought strategies.

The main tools used are:

- Python
- Streamlit
- LLM API
- Pandas for result tables
- Matplotlib for comparison graphs
- Python-dotenv for API configuration

The source code is maintained in a GitHub repository and can be executed using the required Python dependencies.

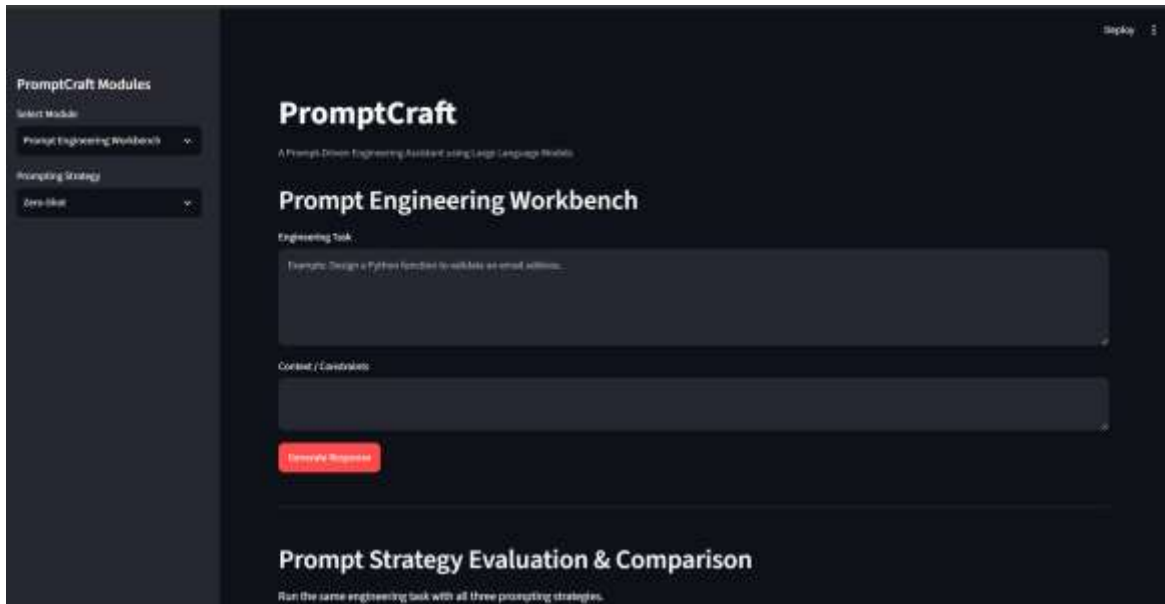


Fig 2: Main Dashboard

5. Results and Validation

The PromptCraft system was validated using sample inputs for each engineering task module. Each module was tested with representative engineering tasks to verify prompt construction, LLM response generation, and output presentation.

Test Case 1 – Code Generation and Explanation

Input:

Generate a Python program to calculate the factorial of a number and explain the algorithm.

Expected Result:

The system should generate valid code with a clear explanation.

Actual Result:

The LLM generated a structured programming solution with an explanation of the implementation.

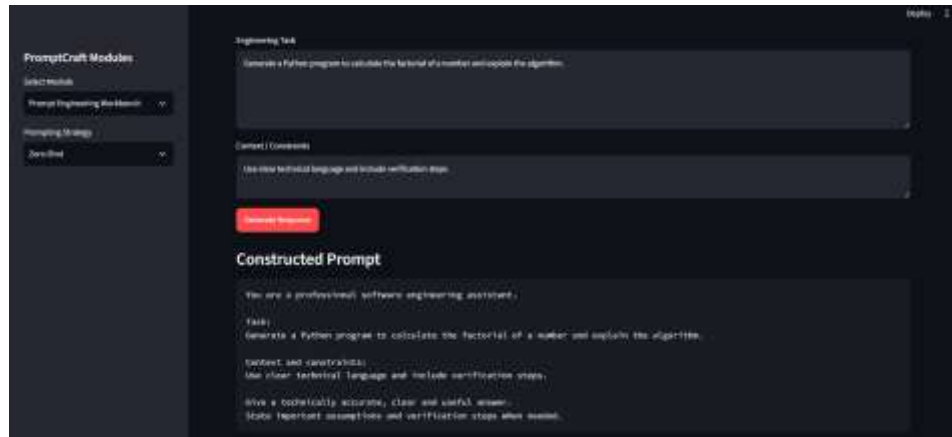


Fig 3: Test Case 1

Test Case 2 – Debugging Assistance

Input:

Identify and correct an error in a program where a variable is used before being initialized.

Expected Result:

The system should identify the error, explain its cause, and provide corrected code.

Actual Result:

The system identified the programming issue and provided an explanation and corrected solution.

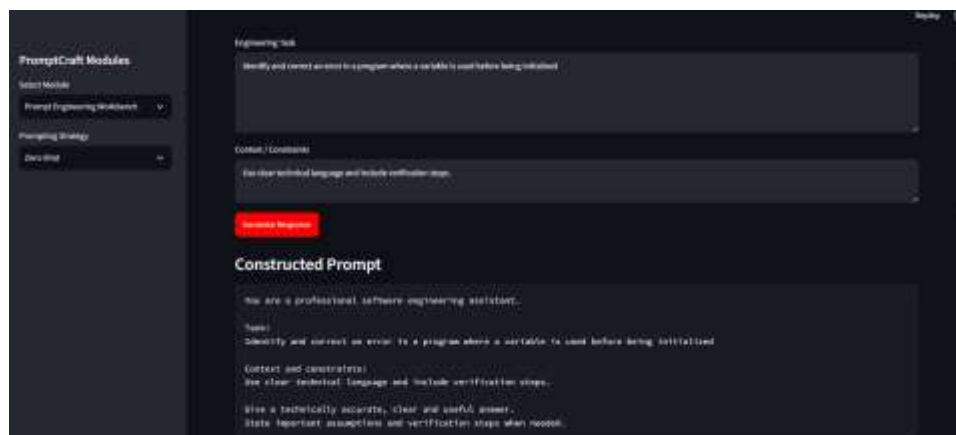


Fig 4: Test Case 2

Test Case 3 – Requirement-to-Documentation

Input:

Create documentation for a student attendance management system.

Expected Result:

The system should generate structured documentation containing requirements, modules, inputs, outputs, and system functions

Actual Result:

The system generated structured technical documentation from the given requirement.

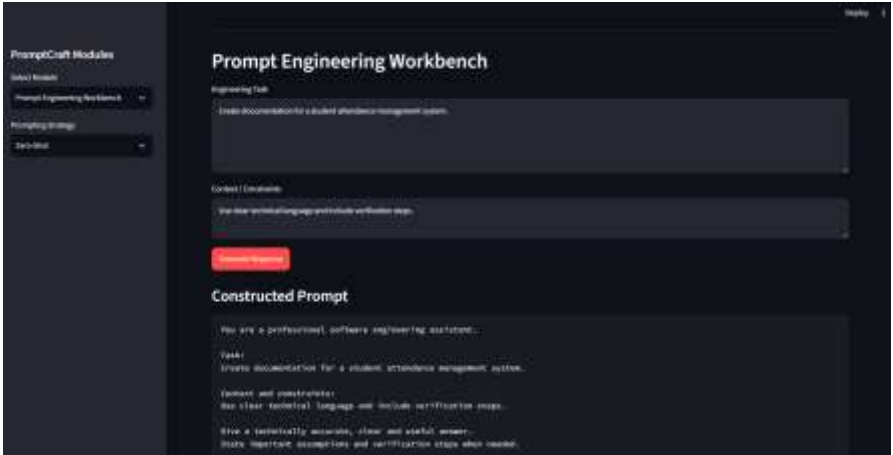


Fig 5: Test Case 3

Prompt Strategy Comparison

The same engineering task was processed using three prompting strategies. The generated responses were compared using response quality, latency, and token usage.

Comparison Table				
Strategy	Quality	Latency (s)	Input Tokens	Total Tokens
Zero-Shot	4	0.25	84	262
Few-Shot	4.4	0.35	114	292
Chain-of-Thought	4.6	0.45	116	294

Fig 6: Comparison Table

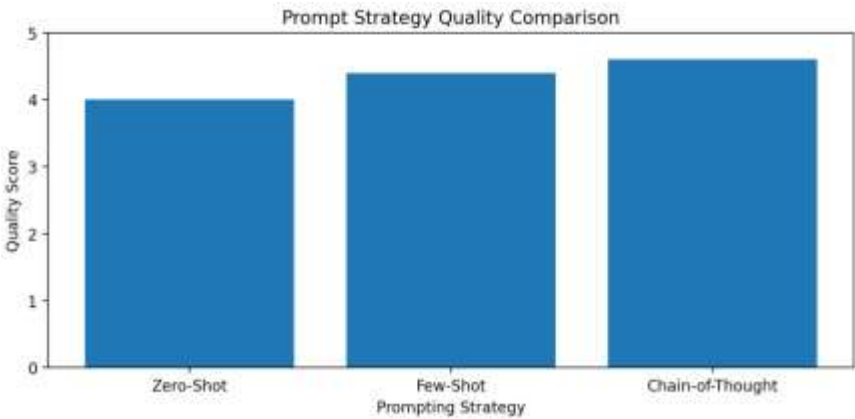


Fig 7 : Comparison Graph

6. Analysis and Engineering Decision

Zero-Shot prompting provides a simple and fast approach because it does not require examples. It is suitable for straightforward engineering tasks where the expected response format is not complex.

Few-Shot prompting provides better control over response structure because examples guide the LLM towards the required format. However, additional examples increase the prompt size and token usage.

Chain-of-Thought prompting is useful for engineering problems requiring systematic analysis and verification. It can produce more structured responses but may require additional processing and tokens.

Based on the comparison, the selection of prompting strategy should depend on task complexity. Zero-Shot is suitable for simple tasks, Few-Shot is useful when a specific response format is required, and Chain-of-Thought is suitable for tasks requiring structured technical reasoning.

7. Broader Considerations

Reliability and Safety:

LLM-generated responses should be reviewed before practical use because generated information may contain errors or hallucinations.

Transparency:

PromptCraft displays the constructed prompt along with the generated response so that users can understand how prompt structure influences the output.

Privacy:

Sensitive personal, academic, or confidential information should not be submitted to external LLM services unless appropriate privacy protections are available.

Reproducibility:

The same prompt templates and test cases can be reused to compare different prompting strategies under similar conditions.

Professional

Responsibility:

The system is intended to assist engineering work rather than replace human verification and decision-making.

8. Conclusion

PromptCraft successfully provides a unified interface for applying prompt engineering techniques to common engineering tasks. The system integrates Zero-Shot, Few-Shot, and Chain-of-Thought prompting with LLM-based code generation, debugging assistance, and requirement-to-documentation generation.

The evaluation module enables comparison of prompting strategies using response quality, latency, and token usage. The application also provides prompt transparency, error handling, and responsible-AI considerations. The main limitation is that LLM responses can vary and may contain incorrect information. Future improvements can include stronger automated evaluation, additional engineering modules, support for multiple LLM providers, and improved response verification.

9. Student Reflection

Through this assignment, I learned how prompt structure affects the quality and usefulness of Large Language Model responses. Implementing different prompting strategies helped me understand the practical difference between Zero-Shot, Few-Shot, and Chain-of-Thought prompting.

I also gained practical experience in integrating an LLM API with Python and developing an interactive Streamlit application. Testing the engineering modules showed the importance of evaluating responses rather than accepting generated content without verification.

If additional time and resources were available, I would improve the system by adding more engineering task modules, stronger automated response evaluation, support for multiple LLM models, and additional test cases using real engineering workloads.

10. References

- [1] OpenAI, *OpenAI API Documentation*, OpenAI Developer Documentation.
- [2] Streamlit, *Streamlit Documentation*, Streamlit.
- [3] Python Software Foundation, *Python Documentation*, Python.
- [4] Pandas Development Team, *Pandas Documentation*.
- [5] Matplotlib Development Team, *Matplotlib Documentation*.
- [6] Vaswani et al., “Attention Is All You Need,” *Advances in Neural Information Processing Systems*, 2017.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course / domain knowledge (Prompt Engineering, LLM Integration, Evaluation)	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques (Python, LLM APIs, Prompt Engineering Frameworks)	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO1/CO2	PO1, PO2	L3	10
Application – Prompt Engineering Techniques	CO1/CO2	PO1, PO2, PO3	L3/L4	8
Application – LLM Task Module Integration	CO1	PO1, PO2, PO3	L3/L4	6
Application – Evaluation & Comparison of Prompts	CO2	PO1, PO2, PO3	L3/L4	6
Solution / Design – Integrated PromptCraft	CO1–CO2	PO2, PO3, PO5	L4/L5/L6	20
Modern tool usage (Python, LLM APIs, Prompt Engineering)	CO1–CO2	PO5	L3	10
Validation of results	CO1–CO2	PO4	L4/L5	15
Analysis & justification / trade-offs	CO1–CO2	PO2, PO4	L4/L5	15
Broader considerations & documentation / reflection	CO1–CO2	PO6, PO12	L3/L4	10

Note: PO numbers indicated are illustrative; faculty shall verify against the current CSA08 CO–PO matrix and map only the POs genuinely assessed.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60 – 69%
Level 1	50 – 59%
Level 0	$< 50\%$

Target CO Attainment: _____ Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____